

JOGO DE DOMINÓ

Tutorial Completo do Código

6 6	5 3	2 4	0 6	1 1
-------	-------	-------	-------	-------

Guia técnico da construção do projeto: estrutura HTML, estilo responsivo, regras, inteligência artificial, turnos, pontuação, arrastar e soltar, mesa em serpentina, sons e armazenamento local.

index.html	style.css	app.js	localStorage
------------	-----------	--------	--------------

HTML • CSS • JavaScript

Conteúdo

Este material foi organizado para permitir uma leitura contínua ou consultas por assunto.

- | | |
|--|--|
| 01. Arquitetura do projeto | 23. Início da partida |
| 02. Mapa dos arquivos | 24. Nova rodada |
| 03. Estrutura do HTML | 25. Turno do jogador |
| 04. Organização visual no CSS | 26. Compra e passe do jogador |
| 05. Responsividade | 27. Turno das IAs |
| 06. Constantes e identificadores | 28. Fim da rodada |
| 07. Mapeamento de áudio | 29. Fim da partida |
| 08. Classe PecaDomino | 30. Renderização geral |
| 09. Classe EstadoRodada | 31. Participantes e mãos ocultas |
| 10. Distribuição das peças | 32. Renderização da mesa |
| 11. Validação dos lados | 33. Peças e pontos visuais |
| 12. Execução da jogada | 34. Arrastar e soltar com Pointer Events |
| 13. Compra e passe | 35. Alvos de encaixe |
| 14. Ordem de turnos e equipes | 36. Histórico e mensagens |
| 15. Encerramento por batida | 37. Modais, regras e estatísticas |
| 16. Bloqueio e pontuação | 38. Sistema de som |
| 17. Inteligência artificial | 39. Algoritmo de serpentina |
| 18. Criação e embaralhamento do conjunto | 40. Funções auxiliares |
| 19. Escolha da abertura | 41. Fluxo técnico completo |
| 20. Armazenamento local | 42. Mapa de funções e linhas |
| 21. Classe DominoApp | 43. Decisões de arquitetura |
| 22. Eventos da interface | |

1. Arquitetura do projeto

O Jogo de Dominó foi construído como uma aplicação web de página única. O navegador carrega a estrutura visual, as regras de estilo e um arquivo JavaScript responsável por toda a lógica da partida. Não é necessário um servidor de aplicação para jogar: a execução ocorre no próprio navegador.

Camada	Responsabilidade
HTML	Cria menu, mesa, cabeçalho, painel lateral, mãos, botões, modal e áreas de status.
CSS	Define identidade visual, filtro, madeira, peças, animações, estados e responsividade.
JavaScript	Implementa regras, estado, IA, turnos, pontuação, arraste, sons, histórico e ranking.
WAV	Fornece efeitos sonoros acionados em momentos específicos.
localStorage	Persiste preferência de som, partidas, estatísticas e ranking no navegador.

2. Mapa dos arquivos

Arquivo	Função no projeto
index.html	Estrutura completa da interface.
style.css	Layout, cores, mesa, peças, responsividade e modais.
app.js	Núcleo de regras, IA, aplicação e renderização.
sons/*.wav	Efeitos de abertura, compra, jogada, passe, erro e resultados.
README.txt	Orientações de execução e descrição dos recursos.

3. Estrutura do HTML

O arquivo index.html separa a aplicação em duas telas principais. A primeira é o menu, identificado por `tela-menu`; a segunda é a mesa, identificada por `tela-jogo`. A classe `screen-active` determina qual delas está visível.

No menu são criados o campo de nome, os cartões de modo, os botões de início, regras, estatísticas e som. Na tela do jogo ficam o cabeçalho com placar, a mesa, as áreas visuais dos participantes, a mão do jogador, o painel de ações e o histórico. Um modal global é reutilizado para regras, estatísticas e resultado final.

```
<section id="tela-menu" class="screen screen-active"> ... </section>
<section id="tela-jogo" class="screen"> ... </section>
```

Por que separar em telas?

A troca entre menu e partida pode ser feita somente alterando classes no DOM. Isso evita recarregar a página e preserva o estado controlado pelo JavaScript.

4. Organização visual no CSS

O CSS utiliza variáveis em `:root` para centralizar cores e medidas recorrentes. O visual combina tons verdes para a mesa e painéis, dourado para destaques e cores claras para as peças. O menu é montado em grid e a área da partida combina cabeçalho, zona da mesa e painel lateral.

Bloco CSS	Linhas aproximadas	Papel
Variáveis globais	3+	Paleta, dimensões e valores reutilizáveis.
Menu e cartões de modo	264-1194	Tela inicial e seleção do modo.
Cabeçalho e placar	1537-1708	Identificação da partida e pontuação.
Mesa e corrente	1789-2261	Feltro, área de peças, extremos e alvos.
Mão e peças	2476-2694	Interação e desenho visual dos dominós.
Painel lateral	2742+	Turno, ações, dicas e histórico.
Modal e avisos	3046-3358	Toast, regras, estatísticas e final.

5. Responsividade

O CSS possui três faixas de adaptação: até 1050 px, até 820 px e até 600 px. Em cada uma, grids e espaçamentos são reorganizados. Em telas mais estreitas, o painel lateral passa para baixo da mesa, o menu usa uma coluna e as peças da mão diminuem para caber melhor.

```
@media (max-width: 1050px) { ... }
@media (max-width: 820px) { ... }
@media (max-width: 600px) { ... }
```

6. Constantes e identificadores

No início de `app.js`, constantes evitam textos soltos espalhados pela lógica. ``LADO_ESQUERDO`` e ``LADO_DIREITO`` representam as pontas da corrente. ``MODULO_MANO_A_MANO`` e ``MODULO_DUPLAS`` identificam o formato da partida. Também existem identificadores para jogador, IA, parceira e IAs laterais, além das duas equipes.

Constante	Uso
LADO_ESQUERDO / LADO_DIREITO	Indicam onde a peça será encaixada.
MODULO_MANO_A_MANO / MODULO_DUPLAS	Controlam distribuição, monte, participantes e interface.
ORDEM_MANO	Sequência jogador -> IA.
ORDEM_DUPLAS	Sequência jogador -> IA direita -> parceira -> IA esquerda.
NOMES	Converte identificadores internos em nomes exibidos na interface.

7. Mapeamento de áudio

O objeto ``AUDIO``, declarado no início de `app.js`, associa nomes lógicos a arquivos WAV. Isso permite chamar ``this.tocar("pedra")`` ou ``this.tocar("erro")`` sem repetir caminhos de arquivo em toda a aplicação.

```
const AUDIO = {
  abertura: "sons/abertura.wav",
  comprar: "sons/comprar.wav",
  pedra: "sons/pedra.wav",
  erro: "sons/erro.wav",
  ...
};
```

8. Classe PecaDomino

A classe ``PecaDomino``, iniciada na linha 278, modela uma peça do jogo. O construtor recebe os dois lados e gera um identificador. Os getters ``total`` e ``dupla`` calculam propriedades importantes sem armazenar valores duplicados.

Membro	Linhas	Função
constructor(a,b,id)	293-319	Armazena os lados e o identificador.
total	333	Retorna a soma dos dois lados.
dupla	347	Informa se os dois lados são iguais.

combinaCom(valor)	355-382	Verifica se a peça encaixa em determinado valor.
outroValor(valor)	390-428	Obtém o lado que ficará aberto após um encaixe.
texto()	440	Gera representação como [6 3].

Essa classe concentra regras próprias da peça. Assim, o restante do sistema não precisa repetir comparações como `a === valor || b === valor` em vários lugares.

9. Classe EstadoRodada

A classe `EstadoRodada`, iniciada na linha 450, concentra tudo o que pertence a uma única rodada: mãos, monte, corrente da mesa, extremos, turno, passes consecutivos, vencedor e motivo do encerramento. Ela funciona como o modelo principal das regras.

```
this.maos = {};
this.monte = [];
this.mesa = [];
this.esquerda = null;
this.direita = null;
this.turno = JOGADOR;
this.passesConsecutivos = 0;
```

10. Distribuição das peças

O método `preparar()`, a partir da linha 630, cria as 28 peças, embaralha o conjunto e distribui conforme o modo. Em Duplas, quatro participantes recebem 7 peças e `monte` fica vazio. No Mano a Mano, jogador e IA recebem 7 cada e `pecas.slice(14)` forma o monte com as 14 restantes.

```
if (modo === MODO_DUPLAS) {
  ORDEM_DUPLAS.forEach((p, indice) => {
    this.maos[p] = pecas.slice(indice * 7, indice * 7 + 7);
  });
  this.monte = [];
} else {
  this.maos[JOGADOR] = pecas.slice(0, 7);
  this.maos[IA] = pecas.slice(7, 14);
  this.monte = pecas.slice(14);
}
```

11. Validação dos lados

O método `ladosValidos(peca)`, iniciado na linha 854, verifica os extremos da mesa e retorna uma lista com os lados em que a peça pode entrar. Quando a mesa está vazia, qualquer lado é aceito para a primeira colocação. Depois disso, cada ponta é testada com `peca.combinaCom(...)`.

Os métodos `possuiJogada()`, `podeComprar()` e `podePassar()` usam essa validação para habilitar ou bloquear ações da interface. Com isso, as regras não ficam apenas nos botões: elas são consultadas pelo estado antes de qualquer operação.

12. Execução da jogada

O método `jogar()`, linhas 1024-1271, é a operação central de encaixe. Ele valida se a rodada está aberta, se a peça realmente pertence à mão informada e se o lado pedido é permitido. Depois orienta a peça de acordo com o extremo, atualiza a corrente e remove a peça da mão.

- **Primeira peça:** é inserida no centro e define ``esquerda`` e ``direita``.
- **Encaixe à esquerda:** usa `outroValor(this.esquerda)`` para descobrir o novo valor aberto.
- **Encaixe à direita:** faz o mesmo com `this.direita``.
- **Após a jogada:** a peça sai da mão, os passes consecutivos voltam a zero e é verificado se alguém bateu.

13. Compra e passe

`comprar()` (1279-1322) só retira uma peça do monte no Mano a Mano. `registrarPasse()` (1330-1379) incrementa `passesConsecutivos`` e verifica se todos os participantes passaram com o monte vazio. Quando o número de passes alcança a quantidade de participantes, `encerrarBloqueio()` é chamado.

14. Ordem de turnos e equipes

`proximoParticipante()` usa a posição do participante dentro de `ordemTurnos`` e avança circularmente com módulo. `equipeDe()` converte participantes em equipe do jogador ou equipe adversária. `totalEquipe()` soma os pontos das mãos correspondentes.

```
return this.ordemTurnos[(indice + 1) % this.ordemTurnos.length];
```

Essa estrutura permite usar o mesmo motor de rodada para dois ou quatro participantes, alterando apenas a lista de turnos e a formação das equipes.

15. Encerramento por batida

`verificarMaoVazia()`, linhas 1607-1701, é executado depois de uma peça ser removida da mão. Se a quantidade chegar a zero, a rodada é marcada como encerrada e o motivo recebe ``bateu``. No Mano a Mano, o próprio participante é o vencedor; em Duplas, o vencedor é a equipe à qual ele pertence.

16. Bloqueio e pontuação

`encerrarBloqueio()`, linhas 1708-1941, calcula os pontos presos nas mãos. Em Duplas, compara `totalEquipe(EQUIPE_JOGADOR)`` com `totalEquipe(EQUIPE_ADVERSARIA)``. No Mano a Mano, compara diretamente a mão do jogador e a mão da IA. Totais iguais produzem empate.

`pontosDaRodada()`, linhas 1948-2104, transforma o resultado em pontos de placar. Em uma batida, são usados os pontos restantes do adversário. Em bloqueio, usa-se a diferença entre os totais. `Math.max(1, pontos)`` assegura pelo menos um ponto quando há vencedor.

17. Inteligência artificial

A classe `InteligenciaDomino`, iniciada na linha 2114, usa uma heurística. O método `escolherJogada()` primeiro conta quantas vezes cada número aparece na mão. Em seguida cria todas as combinações possíveis de peça e lado e atribui uma pontuação.

```
let pontuacao = peca.total * 2;
pontuacao += (frequencias.get(valorAberto) || 0) * 3;
if (peca.dupla) pontuacao += 5;
if (peca.total >= 10) pontuacao += 2;
```

A soma alta ajuda a eliminar peças perigosas; a frequência do valor que ficará aberto favorece continuidade; duplas e peças muito pesadas recebem bônus. Depois, as jogadas são ordenadas e a opção com maior pontuação é escolhida. Esse modelo é simples, determinístico a partir do estado e suficientemente rápido para rodar diretamente no navegador.

18. Criação e embaralhamento do conjunto

`criarConjuntoCompleto()` (2341-2405) usa dois laços: o primeiro vai de 0 a 6 e o segundo começa em `a`. Isso gera cada combinação uma única vez, totalizando 28 peças. `embaralhar()` (2414-2459) aplica o algoritmo Fisher-Yates, trocando cada posição por uma posição aleatória dentro da parte ainda não fixada.

```
for (let a = 0; a <= 6; a += 1) {
  for (let b = a; b <= 6; b += 1) {
    pecas.push(new PecaDomino(a, b));
  }
}
```

19. Escolha da abertura

`melhorPecaInicial()` prioriza duplas e, se houver alguma, escolhe a de maior número. Sem duplas, ordena por soma e depois pelos maiores lados. `chavePecaInicial()` transforma essa preferência em uma chave comparável. `primeiroParticipante()` aplica a lógica a todas as mãos e seleciona o melhor candidato da rodada.

20. Armazenamento local

A classe `ArmazenamentoLocal`, iniciada na linha 2599, encapsula o uso de `localStorage`. `carregar()` tenta ler e converter JSON. `salvarPartida()` adiciona um resultado e limita a coleção a 300 partidas. `partidasDoJogador()` filtra pelo nome sem diferenciar maiúsculas de minúsculas. `ranking()` agrega partidas, vitórias e pontos por nome.

Por que encapsular localStorage?

A aplicação usa uma interface única para persistência. Se a forma de armazenamento mudar no futuro, as regras da partida não precisam ser reescritas; apenas a classe responsável pela persistência precisaria ser substituída.

21. Classe DominoApp

`DominoApp`, iniciada na linha 2860, coordena a aplicação. Ela possui uma instância de `EstadoRodada`, uma de `InteligenciaDomino` e uma de `ArmazenamentoLocal`. Também mantém placares, número da rodada, peça selecionada, bloqueio da interface, timers, estado do som e histórico visual.

```
this.estado = new EstadoRodada();
this.ia = new InteligenciaDomino();
this.storage = new ArmazenamentoLocal();
this.pontosJogador = 0;
this.pontosIA = 0;
```

22. Eventos da interface

`cacheDOM()` (3077-3221) reúne referências de elementos pelo `id`. `bindEventos()` (3228-3670) liga cliques, teclado, drag-and-drop e redimensionamento. Essa etapa transforma o HTML estático em uma interface controlada pelo objeto `DominoApp`.

- **Cartões de modo:** trocam `this.modos`.
- **Botão Iniciar:** chama `iniciarPartida()`.
- **Botões de lado:** chamam `jogarSelecionada()`.
- **Comprar e passar:** chamam os métodos do turno humano.
- **F8:** alterna o som.
- **Resize:** recalcula a mesa durante uma partida.

23. Início da partida

`iniciarPartida()` (3738-3885) valida o nome, limita-o a 40 caracteres, zera placares e rodada, registra o instante inicial, limpa logs, marca a aplicação como em jogo, troca para a tela da mesa e chama `novaRodada()`.

24. Nova rodada

`novaRodada()` (3938-4128) primeiro verifica se alguém já atingiu a meta de pontos. Caso contrário, incrementa o número da rodada, cria um novo `EstadoRodada`, distribui as peças e localiza o participante que possui a melhor abertura. Essa peça é jogada automaticamente no centro antes do primeiro turno normal.

```
const [primeiro, peca] = this.estado.primeiroParticipante();
this.estado.turno = primeiro;
this.estado.jogar(peca, LADO_DIREITO, primeiro);
this.estado.turno = this.estado.proximoParticipante(primeiro);
```

25. Turno do jogador

`prepararTurnoAtual()` libera a interface quando o turno é do jogador e agenda a IA quando não é. `selecionarPeca()` verifica os lados possíveis, atualiza o destaque visual e habilita os alvos. `jogarSelecionada()` valida novamente o lado e somente então chama `estado.jogar()`.

A validação duplicada - na interface e dentro do estado - é importante. Mesmo que um botão ou evento de arraste seja acionado de maneira inesperada, a regra continua protegida no núcleo da rodada.

26. Compra e passe do jogador

``comprarJogador()`` (4612-4824) impede compra quando há jogada possível. Depois de comprar, verifica imediatamente se a nova peça serve e a destaca. ``passarJogador()`` (4831-5021) impede o passe se ainda houver uma jogada ou se o monte contiver peças. Somente após essas verificações registra o passe e avança o turno.

27. Turno das IAs

``turnoIA()`` (5034-5358) pede uma decisão para ``this.ia.escolherJogada()``. Se não houver jogada no Mano a Mano, a IA compra e chama o próprio método novamente depois de um pequeno intervalo. Se não houver jogada nem compra possível, registra um passe. Se houver jogada, executa o encaixe, atualiza interface e histórico e avança o turno.

Os atrasos de centenas de milissegundos são controlados por ``agendar()``. Eles deixam as ações legíveis visualmente sem travar a execução do navegador.

28. Fim da rodada

``tratarFimRodada()`` (5365-5659) lê o vencedor e os pontos calculados no estado. O método atualiza ``pontosJogador`` ou ``pontosIA``, escolhe mensagens e sons adequados, bloqueia novas ações e agenda ``novaRodada()`` após a apresentação do resultado.

29. Fim da partida

``encerrarPartida()`` (5666-5964) é chamado quando a meta de 100 pontos é atingida. Ele calcula a duração, registra a partida em ``ArmazenamentoLocal``, toca o efeito de vitória ou derrota e abre um modal com placar, rodadas e tempo. O modal oferece reinício ou retorno ao menu.

```
this.storage.salvarPartida({
  nome: this.nome,
  dataHora: new Date().toISOString(),
  vencedor: venceu ? "jogador" : "ia",
  pontosJogador: this.pontosJogador,
  pontosIA: this.pontosIA,
  rodadas: this.rodada,
  duracaoSegundos: duracao,
  modo: this.nomeModo()
});
```

30. Renderização geral

``renderTudo()`` (5971-6208) funciona como ponto central de sincronização visual. Ele sincroniza nome, modo, placar, número da rodada, participantes, mesa, mão do jogador, botões, histórico, extremos, monte

e indicação de turno. Em vez de cada regra editar dezenas de elementos, o estado é alterado e depois a tela é reconstruída de forma consistente.

31. Participantes e mãos ocultas

``renderParticipantes()`` (6215-6467) mostra ou oculta os elementos conforme o modo. Em Duplas, calcula a quantidade de peças da parceira e das duas IAs laterais. No Mano a Mano, exibe somente a IA adversária e o monte. ``renderVersos()`` cria pequenos versos de peças sem revelar seus valores.

32. Renderização da mesa

``renderMesa()`` (6474-6756) localiza o índice da peça de abertura, chama ``calcularLayoutMesa()`` e cria um elemento visual para cada peça da corrente. A orientação pode ser horizontal ou vertical. Duplas são sempre desenhadas verticalmente, e peças em linhas invertidas podem ter seus valores visuais trocados para acompanhar o sentido da serpentina.

- **Peça inicial:** recebe a classe ``opening`` e permanece destacada.
- **Peças das pontas:** recebem a classe ``endpoint``.
- **Coordenadas:** são calculadas em porcentagem para acompanhar o tamanho da mesa.
- **Drop targets:** são reposicionados a partir dos próximos pontos calculados para cada ponta.

33. Peças e pontos visuais

``criarElementoPeca()`` (9508-9595) cria duas metades para cada dominó e chama ``desenharPontos()``. Essa função possui uma tabela de coordenadas para os valores 0 a 6. Cada ponto é um ``span`` posicionado com ``left`` e ``top`` em porcentagem.

```
5: [[28, 28], [72, 28], [50, 50], [28, 72], [72, 72]],  
6: [[28, 23], [72, 23], [28, 50], [72, 50], [28, 77], [72, 77]]
```

Essa abordagem dispensa imagens separadas para cada peça e mantém os dominós nítidos em diferentes tamanhos.

34. Arrastar e soltar com Pointer Events

O fluxo moderno de arraste começa em ``iniciarArrastePeca()`` (7104-7317). O ponteiro é capturado, a posição inicial é salva e eventos de movimento e soltura são registrados. ``moverArrastePeca()`` (7325-7526) só considera arraste real depois de uma distância mínima de 6 pixels, evitando confundir um clique com movimento.

Durante o movimento, uma cópia da peça acompanha o ponteiro e o jogo chama ``ladoPeloPonto()`` para descobrir qual extremo está mais próximo. ``finalizarArrastePeca()`` (7534-7708) valida se o ponteiro terminou dentro da mesa e sobre um lado permitido. ``limparArraste()`` remove a cópia e os listeners temporários.

35. Alvos de encaixe

``atualizarDropTargets()`` (8105-8155) mostra somente os alvos válidos para a peça selecionada. ``ladoPeloPonto()`` (8421-8544) calcula o centro de cada alvo com ``getBoundingClientRect()`` e compara as distâncias com ``Math.hypot()``. Quando ambos os lados são permitidos, o menor valor de distância define o encaixe.

36. Histórico e mensagens

``setStatus()`` atualiza a mensagem principal. ``adicionarLog()`` insere uma ação no início do histórico e mantém até 30 itens em memória. ``renderLog()`` mostra os oito eventos mais recentes com horário. ``mostrarToast()`` cria avisos temporários para ações como rodada, passe ou erro.

37. Modais, regras e estatísticas

``abrirRegras()`` monta o conteúdo explicativo do jogo dentro do modal reutilizável. ``abrirEstatisticas()`` consulta o armazenamento, calcula vitórias, derrotas e pontos, monta as últimas partidas e o ranking e depois chama ``abrirModal(html)``. O HTML dinâmico usa ``escapeHTML()`` quando insere nomes ou textos vindos do usuário.

38. Sistema de som

``tocar()`` (aproximadamente 9137-9230) verifica se o som está ativo e se existe arquivo para o nome pedido. Um ``Map`` chamado ``audioCache`` reutiliza objetos ``Audio``, evitando criar um novo carregamento a cada jogada. Antes de tocar, ``currentTime`` volta a zero. ``alternarSom()`` grava a preferência no ``localStorage``.

39. Algoritmo de serpentina

``calcularLayoutMesa()``, iniciado na linha 9758, resolve um dos pontos visuais mais importantes do projeto: colocar uma corrente potencialmente longa dentro da área disponível sem sobrepor linhas. O algoritmo parte do centro e constrói dois ramos independentes, um para a esquerda e outro para a direita.

1. Calcula dimensões mínimas da área e medidas aproximadas da peça.
2. Define limites horizontais seguros e a posição central.
3. Para cada ramo, avança horizontalmente enquanto houver espaço.
4. Ao alcançar a borda, insere um ponto de curva vertical.
5. Aumenta o nível da linha e inverte a direção horizontal.
6. Calcula um espaçamento vertical que caiba na altura disponível.
7. Converte as coordenadas em porcentagem.
8. Retorna posições das peças e os próximos pontos de encaixe da esquerda e direita.

Por que guardar o índice da peça inicial?

A corrente cresce para os dois lados. O índice da abertura permite saber quantas peças pertencem ao ramo esquerdo e quantas pertencem ao ramo direito, mantendo o centro visual estável mesmo após muitas jogadas.

40. Funções auxiliares

Função	Linhas	Uso
renderVersos()	9442-9494	Desenha peças viradas dos participantes controlados pela IA.
criarElementoPeca()	9508-9595	Monta o DOM visual de um dominó.
desenharPontos()	9606-9743	Posiciona os pontos de 0 a 6.
calcularLayoutMesa()	9758-10515	Calcula a corrente em serpentina.
calcularPosicoesSerpentina()	10530-10542	Atalho que retorna apenas as posições.
clamp()	10559	Limita números a um intervalo.
escapeHTML()	10568-10594	Evita inserção direta de caracteres especiais em HTML.
formatarDuracao()	10603-10635	Converte segundos em minutos e segundos.

41. Fluxo técnico completo

1. DOMContentLoaded cria `new DominoApp()`.
2. O construtor guarda dependências, faz cache do DOM e registra eventos.
3. O usuário escolhe o modo e chama `iniciarPartida()`.
4. `novaRodada()` cria `EstadoRodada`, distribui e executa a abertura.
5. `prepararTurnoAtual()` libera jogador ou agenda IA.
6. Uma jogada chama `EstadoRodada.jogar()`, que atualiza corrente, extremos e mão.
7. Depois de cada ação, `tratarFimRodada()` verifica encerramento.
8. Sem encerramento, `proximoParticipante()` define o próximo turno.
9. Batida ou bloqueio leva ao cálculo de `pontosDaRodada()`.
10. O placar é atualizado e uma nova rodada começa.
11. Ao chegar a 100 pontos, `encerrarPartida()` salva o resultado e abre o modal final.

42. Mapa de funções e linhas

Elemento	Linhas	Responsabilidade
PecaDomino	278-444	Modelo de uma peça.
EstadoRodada	450-2108	Estado e regras de uma rodada.
preparar()	630+	Distribuição e reinicialização.
ladosValidos()	854+	Lados em que uma peça pode entrar.
jogar()	1024-1271	Encaixe e recálculo dos extremos.
comprar()	1279-1322	Compra no Mano a Mano.
registrarPasse()	1330-1379	Passe e detecção de bloqueio.
primeiroParticipante()	1427-1492	Escolha de abertura.
encerrarBloqueio()	1708-1941	Resultado de mesa fechada.
pontosDaRodada()	1948-2104	Cálculo da pontuação.
InteligenciaDomino	2114-2333	Heurística da IA.
criarConjuntoCompleto()	2341-2405	Geração das 28 peças.
embaralhar()	2414-2459	Fisher-Yates.
ArmazenamentoLocal	2599-2854	Histórico e ranking.
DominoApp	2860-9431	Orquestração da interface e partida.
bindEventos()	3228-3670	Eventos de clique, teclado e arraste.
iniciarPartida()	3738-3885	Inicialização do placar e da tela.
novaRodada()	3938-4128	Nova distribuição e abertura.
selecionarPeca()	4246-4416	Seleção e lados válidos.
jogarSelecionada()	4424-4605	Jogada humana.
comprarJogador()	4612-4824	Compra humana.
passarJogador()	4831-5021	Passe humano.
turnoIA()	5034-5358	Turno automático.
tratarFimRodada()	5365-5659	Aplica pontos e agenda próxima rodada.

encerrarPartida()	5666-5964	Salva resultado e mostra final.
renderTudo()	5971-6208	Sincronização visual geral.
renderMesa()	6474-6756	Desenha a corrente.
iniciarArrastePeca()	7104-7317	Início do Pointer Drag.
moverArrastePeca()	7325-7526	Movimento da peça arrastada.
finalizarArrastePeca()	7534-7708	Validação do drop.
ladoPeloPonto()	8421-8544	Escolha do extremo mais próximo.
abrirRegras()	8861-8885	Modal de regras.
abrirEstatisticas()	8893-9029	Histórico e ranking.
criarElementoPeca()	9508-9595	DOM de um dominó.
desenharPontos()	9606-9743	Pontos visuais.
calcularLayoutMesa()	9758-10515	Serpentina e curvas.

43. Decisões de arquitetura

- **Estado separado da interface:** `EstadoRodada` pode validar e executar regras sem depender diretamente do HTML.
- **IA separada:** a escolha automática fica em uma classe própria e recebe somente o estado e a mão.
- **Persistência separada:** `ArmazenamentoLocal` concentra leitura, gravação e ranking.
- **Renderização centralizada:** `renderTudo()` evita divergências entre o estado interno e o que aparece na tela.
- **Layout matemático:** a corrente não depende de posições fixas; ela é recalculada pela quantidade de peças e dimensões reais.
- **Eventos com proteção de regra:** botões podem ficar desabilitados, mas o núcleo também valida a ação antes de alterar o estado.
- **Sem imagens para as peças:** os dominós são desenhados com HTML e CSS, inclusive os pontos, mantendo nitidez e flexibilidade.
- **Timers controlados:** `agendar()` registra os timeouts para que possam ser cancelados ao retornar ao menu ou reiniciar.

Leitura recomendada do código

Para compreender o projeto de ponta a ponta, comece por `PecaDomino` e `EstadoRodada`, depois leia `InteligenciaDomino`, em seguida `DominoApp` e finalize com `calcularLayoutMesa()`. Essa ordem acompanha a dependência natural entre regras, decisões, interface e posicionamento visual.